# Correlation Under Non-Linearity

2026-10-21

## Compute Correlation Test P-Value

The modified function allows you to specify the null value of the correlation while also utilizing the p-value mechanism we like, with the ability to specify what kind of p-value we want to be returned. We should refer to the prior version of this file to compare the calculation of the p-value (didn't previously divide by 2).

```
cor.test2 <- function(x, y, r.null, p.out){

  r <- cor(x, y)
  n <- length(x)

  if (r.null == 0){
    stat <- r*sqrt((n-2)/(1-r^2)) # ~ t_{n-2}
    pval.two <- 2*min(pt(stat, n-2, lower.tail = TRUE),
                      pt(stat, n-2, lower.tail = FALSE))
  } else {
    stat <- (atanh(r) - atanh(r.null))/(1/sqrt(n-3)) # approx. ~ N(0,1)
    pval.two <- 2*min(pnorm(stat, lower.tail = TRUE),
                      pnorm(stat, lower.tail = FALSE))
  }

  if (p.out == "two") return(pval.two)

  if (stat < 0){
    pval <- pval.two/2
  } else{
    pval <- 1-pval.two/2
  }

  return(pval)
}
```

## Simulation Function:

This was slightly vibe-coded - I had given a prompt to modify the function to make it take various input and have better overall style.

```
simCorr <- function(n, rho = 0.6, p.out = c("two", "signed")) {

  p.out <- match.arg(p.out)

  if (n < 4)
    stop("n must be at least 4.")

  if (!is.finite(rho) || rho == 0 || abs(rho) >= 1)
    stop("rho must be nonzero and strictly between -1 and 1.")

  # 1. Independent, jointly normal variables
  dat.null <- mvrnorm(
    n, mu = c(0, 0), Sigma = diag(2)
  )

  # 2. Independent variables, one squared
  dat.null.indep.sqr <- dat.null
  dat.null.indep.sqr[, 2] <- dat.null.indep.sqr[, 2]^2

  # 3. Nonlinear dependence: X versus X^2
  x <- rnorm(n)
  dat.null.nonlin <- cbind(x, x^2)

  # 4. Correlated, jointly normal variables
  Sigma.rho <- matrix(
    c(1, rho, rho, 1),
    nrow = 2,
    byrow = TRUE
  )

  dat.corr <- mvrnorm(
    n, mu = c(0, 0), Sigma = Sigma.rho
  )

  # 5. Correlated normal variables, then square Y
  dat.corr.sqr <- dat.corr
  dat.corr.sqr[, 2] <- dat.corr.sqr[, 2]^2

  # Calculate p-values under the appropriate null hypotheses
  pvals <- c(
```

```
    null = cor.test2(
      dat.null[, 1], dat.null[, 2],
      r.null = 0, p.out = p.out
    ),

    null.indep.sqr = cor.test2(
      dat.null.indep.sqr[, 1], dat.null.indep.sqr[, 2],
      r.null = 0, p.out = p.out
    ),

    null.nonlin = cor.test2(
      dat.null.nonlin[, 1], dat.null.nonlin[, 2],
      r.null = 0, p.out = p.out
    ),

    corr = cor.test2(
      dat.corr[, 1], dat.corr[, 2],
      r.null = rho, p.out = p.out
    ),

    corr.sqr = cor.test2(
      dat.corr.sqr[, 1], dat.corr.sqr[, 2],
      r.null = 0, p.out = p.out
    )
  )

  return(pvals)
}
```

**Running The Simulation:** Sample code used to execute the above functions.

```
numSims <- 10000
sampSize <- 2500

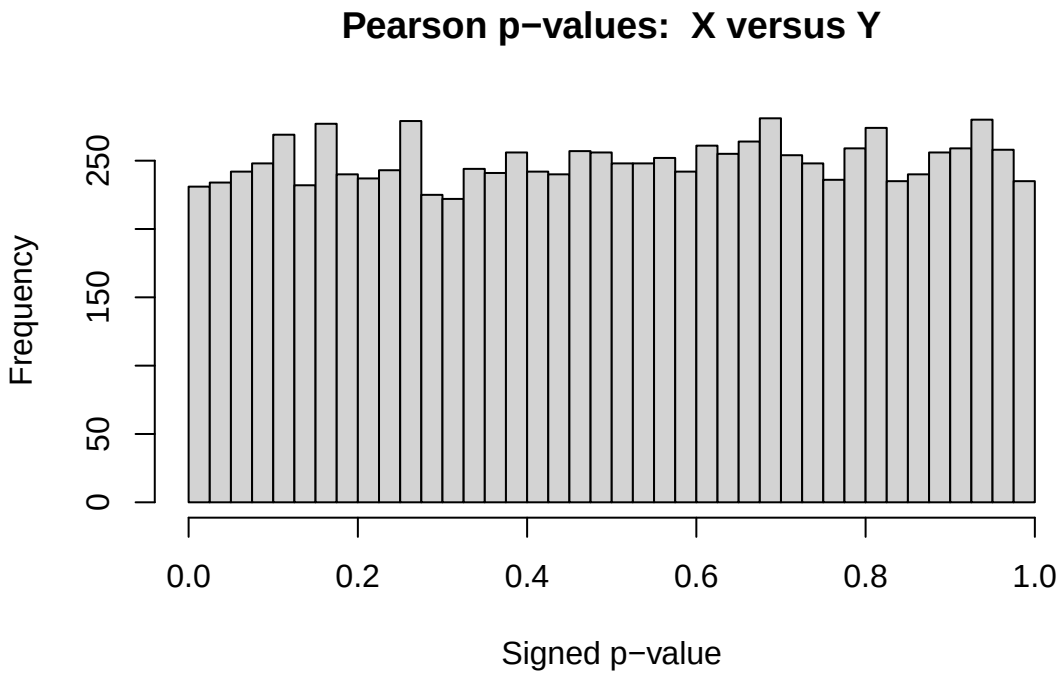
set.seed(123)

pvals.two <- replicate(
  numSims,
  simCorr(sampSize, rho = 0.6, p.out = "two")
)

pvals.signed <- replicate(
  numSims,
  simCorr(sampSize, rho = 0.6, p.out = "signed")
)
```

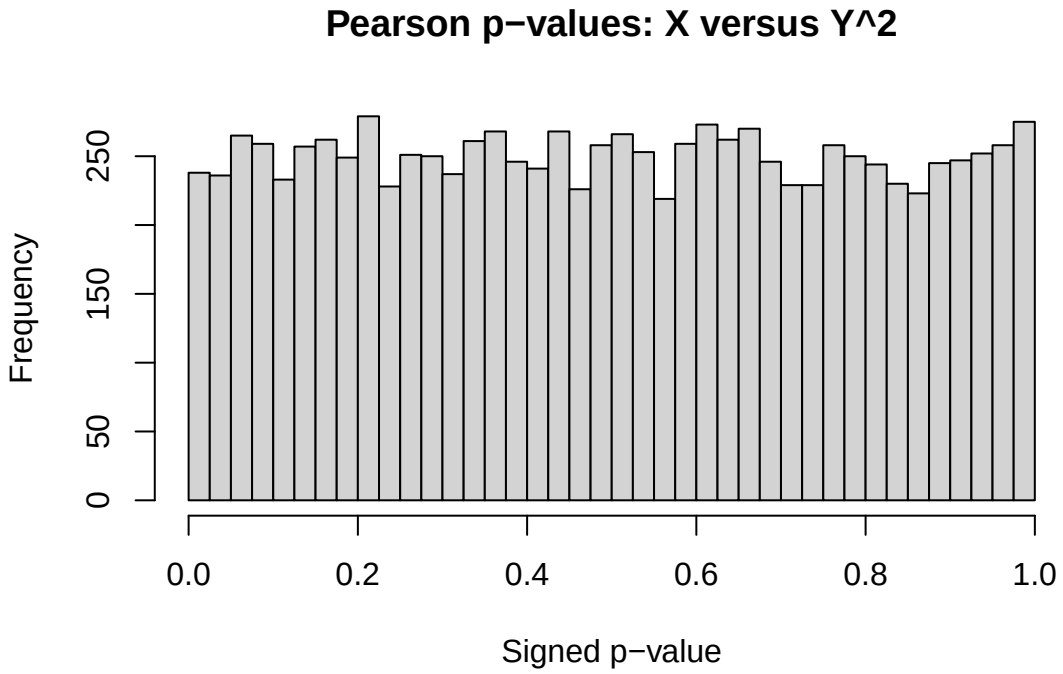
**Case 1: Independent, jointly normal variables**

```
hist(
  pvals.signed["null", ],
  breaks = seq(0, 1, length.out = 41),
  main = "Pearson p-values:  X versus Y",
  xlab = "Signed p-value"
)
```



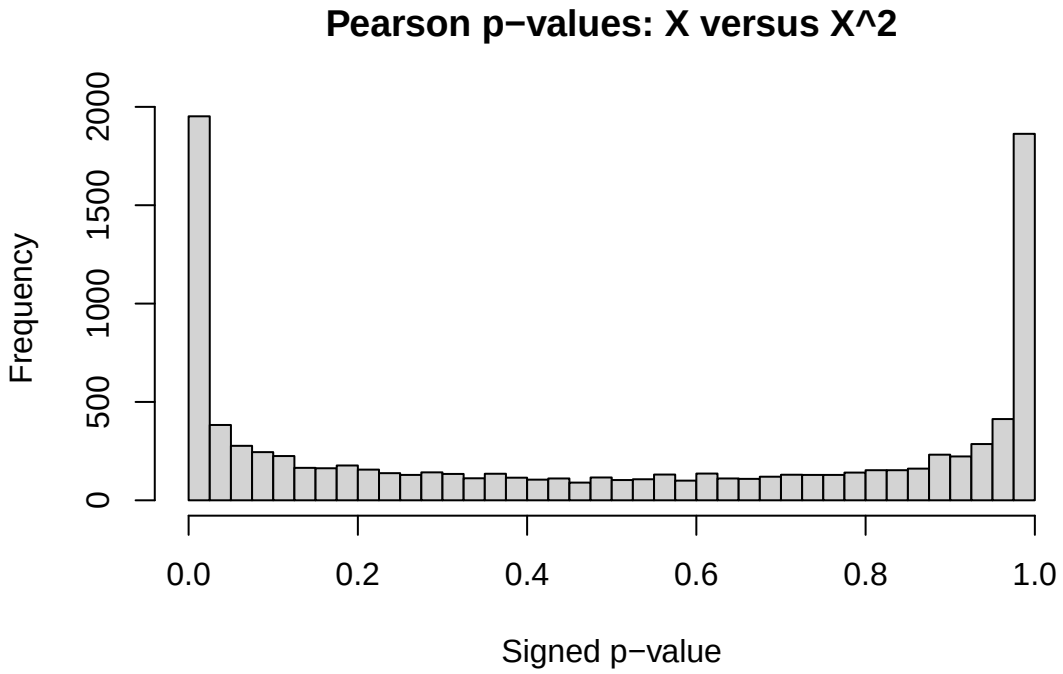
**Case 2: Independent variables, one squared**

```
hist(
  pvals.signed["null.indep.sqr", ],
  breaks = seq(0, 1, length.out = 41),
  main = "Pearson p-values: X versus Y^2",
  xlab = "Signed p-value"
)
```



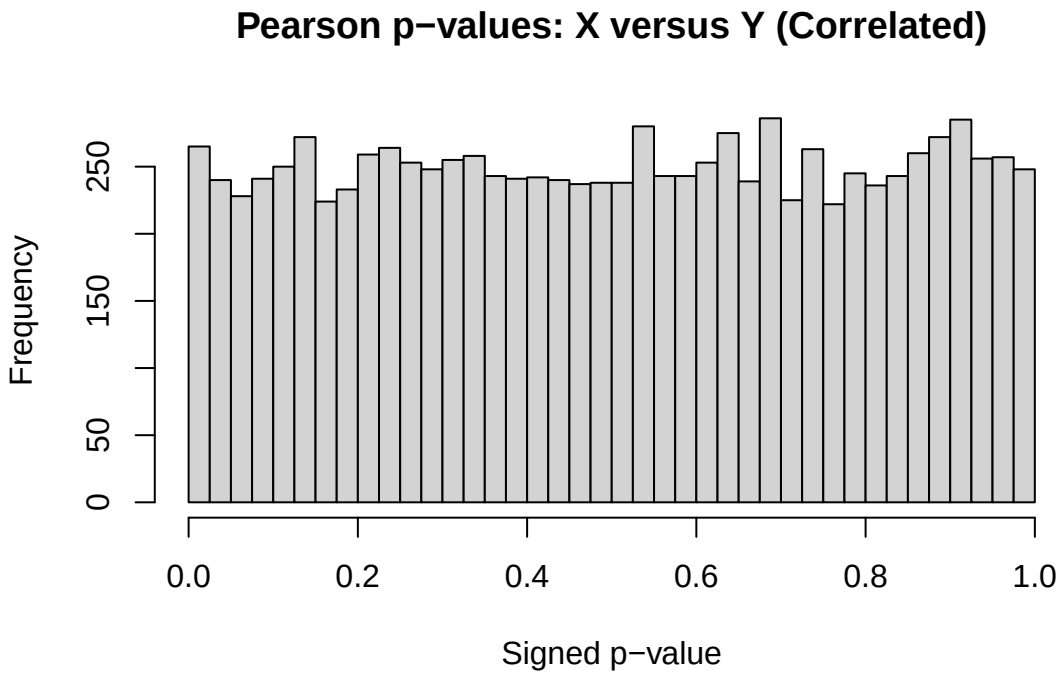
**Case 3: Nonlinear dependence: X versus X^2**

```
hist(
  pvals.signed["null.nonlin", ],
  breaks = seq(0, 1, length.out = 41),
  main = "Pearson p-values: X versus X^2",
  xlab = "Signed p-value"
)
```



**Case 4: Correlated, jointly normal variables**

```
hist(
  pvals.signed["corr", ],
  breaks = seq(0, 1, length.out = 41),
  main = "Pearson p-values: X versus Y (Correlated)",
  xlab = "Signed p-value"
)
```



**Case 5: Correlated normal variables, then square Y**

```
hist(
  pvals.signed["corr.sqr", ],
  breaks = seq(0, 1, length.out = 41),
  main = "Pearson p-values: X versus Y^2 (Correlated)",
  xlab = "Signed p-value"
)
```

